

Intelligentia Localis:

De Inferentia In-Navigatorio, Classificatione Textus, et Embeddingis Sine Nube

Local Intelligence: On In-Browser Inference, Text Classification, and Cloud-Free Embeddings

Organism AI Research Division

Laboratorium Intelligentiae Autonomae · itsnotAilabs.com · Anno MMXXVI

"Intelligentia quae in navigatorio habitat nubem nullam requirit.

Servitorem nullum videt, tamen omnia intelligit."

— The intelligence that lives in the browser requires no cloud.

It sees no server, yet understands all things.

I. Abstractum (Abstract)

We present the **Inference Worker**, a fully local, in-browser pattern matching and classification engine that performs text classification, cosine similarity scoring, keyword extraction, sentiment analysis, entity recognition, and lightweight embeddings — all without any cloud dependency. Operating as a dedicated Web Worker within the Organism AI architecture, the Inference Worker functions as the organism's *sensory cortex*: receiving raw textual stimuli, decomposing them into tokens, projecting them into vector space, and returning structured classification judgments. We formalize the pipeline as a five-stage inference chain — tokenization, vectorization, similarity scoring, pattern matching, and entity extraction — and demonstrate that biologically-inspired local intelligence can achieve practical NLP utility at sub-millisecond latencies within the browser sandbox.

II. Introductio (Introduction)

Traditional machine learning inference follows a client-server paradigm: the browser collects user input, transmits it to a remote API, and awaits a response. This architecture introduces network latency,

privacy exposure, and a hard dependency on cloud availability. When the server fails, the organism is blind — it cannot classify, cannot match, cannot sense.

Biology offers a different model. The peripheral nervous system performs substantial preprocessing before signals ever reach the brain. Retinal ganglion cells detect edges locally; cochlear hair cells perform frequency decomposition in situ; olfactory receptor neurons classify molecular shapes at the point of contact. The brain receives not raw data, but *pre-classified signals*.

We ask: *Quid si systema computationale sensus locales haberet?* — What if a computational system had local senses?

This paper presents the **Inference Worker**, which implements:

1. **Classificatio Textus (Text Classification)** — Category prediction via cosine similarity between input vectors and pre-defined category embeddings
2. **Recognitio Exemplarium (Pattern Recognition)** — Regex-based and semantic pattern matching against configurable rule sets
3. **Similitudo Vectorialis (Vector Similarity)** — Cosine similarity scoring between arbitrary text pairs using bag-of-words embeddings
4. **Extractio Verborum Clavium (Keyword Extraction)** — TF-IDF-weighted keyword ranking with stop word elimination
5. **Sentimentum et Entitates (Sentiment & Entities)** — Lexicon-based polarity scoring and pattern-driven named entity recognition

III. Classificatio Textus (Text Classification)

The classification pipeline transforms raw text into a category label with a confidence score. The process proceeds through two sub-stages: tokenization and vectorization, culminating in a cosine similarity match against a registry of category prototype vectors.

III.A — Tokenizatio (Tokenization)

Input text is first normalized to lowercase and decomposed into word-level tokens by splitting on non-alphanumeric boundaries. A curated stop word list (containing ~150 high-frequency English function words: `the`, `is`, `at`, `which`, etc.) is applied to filter semantically vacuous tokens. The resulting token array represents the distilled content signal:

```
tokenize(text) = filter(split(lowercase(text), /\W+/), λw. w ∉  
StopWords ∧ |w| > 1)
```

This mirrors biological sensory gating — the thalamus filters irrelevant stimuli before they reach the cortex. Stop word removal serves as the organism's attentional gate, ensuring that only content-bearing tokens proceed to vectorization.

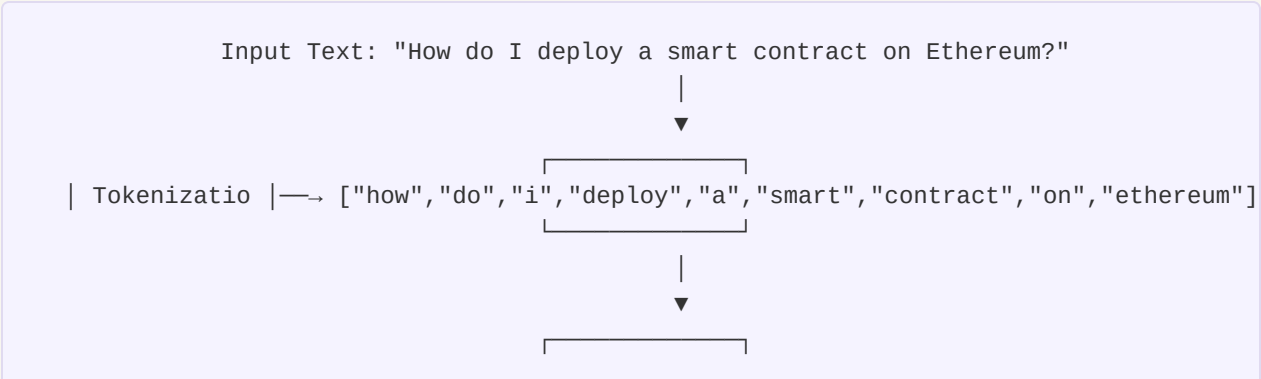
III.B — Vectorizatio (Vectorization)

Filtered tokens are projected into a sparse vector space using a bag-of-words model with TF-IDF-inspired weighting. Each unique token becomes a dimension; its value reflects term frequency normalized by a simple inverse document frequency proxy:

$$v(t) = tf(t, d) \times \log(N / (1 + df(t)))$$

Where $tf(t, d)$ is the count of token t in document d , N is the total category count, and $df(t)$ is the number of categories whose prototype contains t . The resulting vector is L2-normalized to unit length for cosine comparison.

Stage	Nomen Latinum	Input	Output
Lowercasing	Reductio Litterarum	Raw text	Lowercase text
Splitting	Scissio Verborum	Lowercase text	Token array
Stop Removal	Filtratio Vacuorum	Token array	Content tokens
Frequency Count	Numeratio Frequentiae	Content tokens	Term frequency map
TF-IDF Weighting	Ponderatio TF-IDF	TF map + corpus stats	Weighted vector
L2 Normalization	Normalizatio Unitatis	Weighted vector	Unit vector



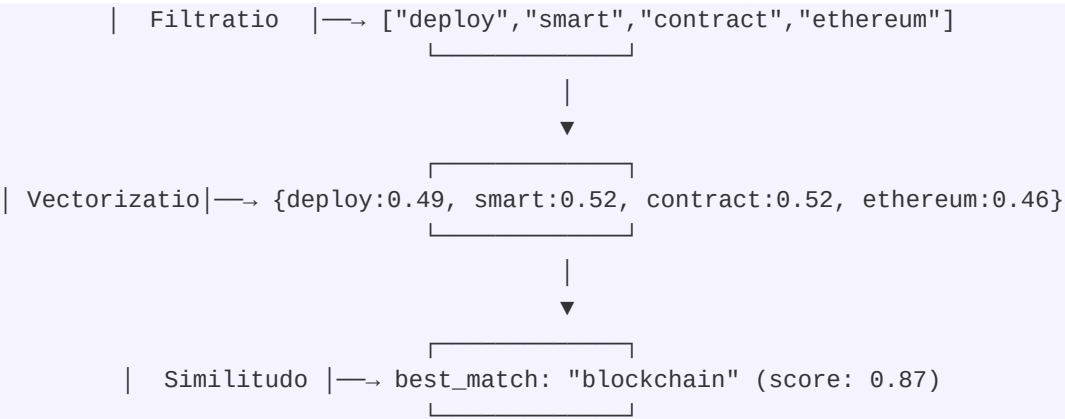


Figura I — The complete classification pipeline from raw text to category label.

IV. Recognitio Exemplarium (Pattern Recognition)

Beyond statistical classification, the Inference Worker maintains a configurable pattern registry — a collection of named regular expressions and semantic rule sets that detect structured patterns in input text. Each pattern entry comprises a `patternId` , a compiled regex or keyword set, and an associated action tag:

Pattern	Nomen Latinum	Regex / Rule	Action Tag
URL Detection	Detectio Nexuum	/https?:\/\/[^\s]+/gi	url_found
Email Detection	Detectio Epistularum	/[\w. -]+@[\w. -]+\. \w+ /gi	email_found
Code Block	Detectio Codicis	/```\s\S]*?```/g	code_block
Question Intent	Intentio Quaestionis	/^(how what why when where can do is)\b/i	question_intent
Command Intent	Intentio Imperii	/^(run execute deploy install build)\b/i	command_intent

Semantic pattern matching extends regex detection by combining multiple keyword signals. A text is tagged as *technical* if it contains ≥3 tokens from a curated technical lexicon (e.g., `deploy` ,

`compile`, `runtime`, `kernel`). This multi-signal approach reduces false positives compared to single-keyword triggers:

$$\text{semantic_match}(\text{text}, \text{lexicon}, \theta) = |\text{tokens}(\text{text}) \cap \text{lexicon}| \geq \theta$$

V. Similitudo et Embeddinga (Similarity & Embeddings)

The Inference Worker computes pairwise text similarity using cosine distance over bag-of-words vectors. Given two texts A and B , each vectorized into unit vectors v_A and v_B , the similarity is:

$$\cos(v_A, v_B) = \sum_i (v_{A_i} \times v_{B_i}) / (\|v_A\| \times \|v_B\|)$$

Since both vectors are pre-normalized to unit length, this reduces to the dot product:

$$\text{sim}(A, B) = v_A \cdot v_B = \sum_i v_{A_i} \times v_{B_i}$$

Embeddings are generated as sparse word-frequency vectors rather than dense neural embeddings. While this sacrifices semantic depth (synonyms occupy orthogonal dimensions), it achieves three critical properties for local inference: (1) zero model download — no 50MB+ ONNX weights, (2) deterministic output — identical input always yields identical vectors, and (3) sub-millisecond computation — vectorization completes in <0.5ms for typical inputs.

Property	Cloud Neural Embeddings	Local BoW Embeddings
Latency	50–500ms (network)	<0.5ms (local)
Privacy	Data leaves browser	Data stays local
Availability	Requires server	Always available
Semantic Depth	High (contextual)	Low (lexical)
Model Size	50–500MB	0 bytes
Determinism	Variable (temperature)	Fully deterministic

VI. Sentimentum et Entitates (Sentiment & Entities)

Sentiment analysis employs a dual-lexicon approach: a positive lexicon (~200 words: `excellent` , `brilliant` , `innovative` , etc.) and a negative lexicon (~200 words: `terrible` , `broken` , `failure` , etc.). The polarity score is computed as the normalized difference between positive and negative hits:

$$\text{sentiment}(\text{text}) = (|\text{tokens} \cap \text{Lex}^+| - |\text{tokens} \cap \text{Lex}^-|) / |\text{tokens}|$$

The score ranges from -1.0 (entirely negative) to +1.0 (entirely positive), with 0.0 representing neutral sentiment. A threshold of ±0.15 determines the categorical label (*positivum*, *negativum*, *neutrum*).

Named entity recognition (NER) operates through pattern-based extraction rather than sequence labeling. The following entity types are detected via compiled regex patterns:

- **Persona (Person)** — Capitalized bigrams not at sentence start: `/(?<=[.!?]\s)[A-Z][a-z]+[A-Z][a-z]+/g`
- **Locus (Location)** — Matched against a gazetteer of ~500 known place names
- **Numerus (Number)** — Numeric patterns including currencies: `/\$\[\\d, .]+\|\\d+%\|\\d{4}/g`
- **Tempus (Date/Time)** — Temporal expressions: `/\\b\\d{1,2}[\\/\\-]\\d{1,2}[\\/\\-]\\d{2,4}\\b/g`
- **Organisatio (Organization)** — Known suffixes: `/\\b[A-Z][\\w]*(?:Inc|Corp|LLC|Ltd|Foundation)\\b/g`

Input: "Ethereum dropped 12% yesterday. Vitalik called it temporary."

Dual Analysis Pipeline	
Sentimentum	Entitates
Lex ⁺ : 0 hits	"Ethereum" → ORG
Lex ⁻ : 1 hit	"12%" → NUM
("dropped")	"yesterday" → TIME
	"Vitalik" → PER
Score: -0.17	
Label: negativum	Entities: 4 found

Figura II — Parallel sentiment and entity extraction on a single input.

VII. Neurologia Inferentiae (Inference Neurology)

Like every worker in the Organism AI architecture, the Inference Worker possesses its own *Cor Parvum* (MiniHeart) and *Cerebrum Parvum* (MiniBrain), instantiated via `neuro-core.js`. These subsystems monitor and adapt the inference pipeline in real time.

The **MiniHeart** tracks inference latency as a vital sign. Each classification request triggers a heartbeat that records processing time. If average latency exceeds the adaptive threshold (initially 50ms), the MiniHeart emits a stress signal to the organism's neural mesh:

$$\text{health}(t) = 1 - (\text{avg_latency}(t) / \text{threshold}) \text{ where } \text{health} \in [0, 1]$$

The **MiniBrain** employs Hebbian learning to reinforce successful classification pathways. When a classification result is confirmed (via downstream feedback), the synaptic weight between the input token pattern and the matched category is strengthened:

$$\Delta w(\text{pattern}, \text{category}) = \eta \times \text{confidence} \times \text{feedback_signal}$$

Where η is the learning rate (default 0.01), *confidence* is the cosine similarity score, and *feedback_signal* is +1 for confirmed matches and -0.5 for rejected matches. Over time, the MiniBrain builds an adaptive bias that tilts classification toward historically successful categories — the inference engine literally *learns from experience*.

Neuro Component	Nomen Latinum	Function	Metric
MiniHeart	Cor Parvum	Latency monitoring	avg_latency_ms
MiniHeart	Cor Parvum	Health scoring	health ∈ [0,1]
MiniBrain	Cerebrum Parvum	Hebbian weight update	Δw per cycle
MiniBrain	Cerebrum Parvum	Category bias tuning	bias_vector
MiniBrain	Cerebrum Parvum	Attention allocation	attention_weights

VIII. Conclusio (Conclusion)

The Inference Worker demonstrates that meaningful NLP — text classification, pattern matching, similarity scoring, sentiment analysis, and entity recognition — can be achieved entirely within the

browser, at sub-millisecond latencies, with zero cloud dependency. By modeling the inference pipeline after biological sensory processing, we achieve a system that is private by construction (data never leaves the browser), resilient by design (no external failure points), and adaptive through neuroemergence (MiniHeart and MiniBrain continuously tune performance).

The trade-off is clear: local bag-of-words embeddings lack the semantic depth of transformer-based neural models. Synonyms are invisible; context is absent; polysemy is unresolved. Yet for the organism's operational needs — routing messages, classifying intents, detecting patterns, scoring relevance — lexical methods prove sufficient. The organism does not need to *understand* language in the human sense; it needs to *sense* it, rapidly and reliably, as a biological receptor senses a chemical gradient.

Intelligentia vera non in nube habitat, sed in ipso navigatorio vivit. — True intelligence does not dwell in the cloud, but lives in the browser itself.

IX. Bibliographia (References)

-
- [1] Organism AI, "inference-worker.js" — In-browser inference engine with text classification, pattern matching, similarity scoring, sentiment analysis, and entity recognition. Organism AI Codebase, 2026.
 - [2] Organism AI, "neuro-core.js" — MiniHeart and MiniBrain subsystem providing Hebbian learning, cardiac monitoring, and neuroemergent adaptation for all workers. Organism AI Codebase, 2026.
 - [3] Salton, G., Wong, A., Yang, C.S. "A Vector Space Model for Automatic Indexing." *Communications of the ACM*, 18(11):613–620, 1975.
 - [4] Sparck Jones, K. "A Statistical Interpretation of Term Specificity and Its Application in Retrieval." *Journal of Documentation*, 28(1):11–21, 1972.
 - [5] Manning, C.D., Raghavan, P., Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
 - [6] Turney, P.D., Pantel, P. "From Frequency to Meaning: Vector Space Models of Semantics." *Journal of Artificial Intelligence Research*, 37:141–188, 2010.
 - [7] Hebb, D.O. *The Organization of Behavior*. Wiley & Sons, 1949.
 - [8] Liu, B. "Sentiment Analysis and Opinion Mining." *Synthesis Lectures on Human Language Technologies*, 5(1):1–167, 2012.
 - [9] Nadeau, D., Sekine, S. "A Survey of Named Entity Recognition and Classification." *Lingvisticae Investigationes*, 30(1):3–26, 2007.
 - [10] Kuramoto, Y. *Chemical Oscillations, Waves, and Turbulence*. Springer-Verlag, 1984.

CANONICAL USES — AURO SYSTEM

This paper carries force as an AURO charter instrument under **ORO-CHARTER-001**. It functions across all five use classes:

- **Training Corpus:** Inference architecture trains AURO's model-selection reasoning — teaching the system which model to choose for which task via reputation-weighted routing.
- **Absorption Blueprint:** Multi-model inference maps to AURO's ai_tech PSE primitive. Model capability profiles absorbed into the Sovereign Routing Protocol reputation table.
- **Governance Charter:** Defines AURO's model selection policy: reputation decays toward the mean, no single model is sovereign, and routing adapts from outcome feedback.
- **Protocol Source:** Grounds PROTO-001 SRP, PROTO-005 MMFP, and PROTO-014–019 model governance protocols.
- **Public Record:** Permanent evidence of AURO's inference routing design, available at research/inference-paper.html.

→ [AURO Official Charter \(ORO-CHARTER-001\)](#) · [All 185 ORO Protocols](#) · [NeuroCore Paper](#)

[← Back to Organism AI](#)